10-Day Full Stack + AI/ML Production Project Development Handbook

Portfolio goal: Build 10 professional, business-oriented software applications.

Target: Demonstrate the skills of a Full Stack Developer, AI/ML Engineer, Generative AI Engineer, Agentic AI Developer and Automation Engineer.

Development schedule: One project per day.

MVP development time: 3 hours per project.

Commercial target: ₹1 lakh+ potential implementation value per customised business deployment. This is a target for a validated commercial solution, not a guaranteed price for a three-hour MVP.

Part 1 — Common Development Standard

Every project must follow these mandatory requirements.

1. Professional landing page

Product name and logo; hero section; business problem and solution; features and benefits; how it works; pricing section where appropriate; FAQ section; contact page; login and registration.

2. Separate user interfaces

Create the relevant interfaces for Admin, Employee or staff, Customer, and Manager or supervisor (if needed). Each role must have appropriate permissions and a relevant dashboard.

3. Complete CRUD functionality

Implement Create, Read, Update and Delete operations for the important business entities (for example: create, view, update and delete or archive products). Use confirmation before destructive operations.

4. Authentication and authorization

Registration, login, logout, password recovery, secure password storage, role-based access control, protected routes and session management.

5. Relational database

Use PostgreSQL as the primary database. Requirements: proper schema, primary and foreign keys, relationships, constraints, indexes where useful, database migrations and data validation.

6. AI/ML integration

AI must work with actual application data. Do not build a disconnected chatbot just to demonstrate AI.

Requirements: AI model integration; backend-controlled AI operations; structured outputs where appropriate; error handling; input validation; model evaluation; human approval for consequential operations.

7. Live deployment

Frontend deployment, backend deployment, database hosting, environment variables, production configuration, a working demonstration account and basic monitoring.

8. GitHub documentation

Every repository must include: README.md, project overview, business problem, features, technology stack, architecture diagram, data-flow diagram, database ER diagram, API documentation, installation instructions, environment variable examples, testing instructions, screenshots and demo URL.

Additional engineering requirements

Automated tests; logging; error handling; Docker configuration; CI/CD pipeline; security checks; database backup strategy; input validation; API rate limiting where appropriate.

Part 2 — Technology Stack

Use a consistent stack across the projects so that you spend your time learning architecture and building products instead of repeatedly learning new frameworks.

Layer	Technology
Frontend	Next.js + TypeScript
Styling	Tailwind CSS
UI components	shadcn/ui
Backend	Python + FastAPI
Database	PostgreSQL
ORM	SQLAlchemy
Authentication	Secure session or JWT-based authentication
AI	OpenAI API or another suitable model API
RAG	pgvector
ML	Scikit-learn, XGBoost, PyTorch
Computer Vision	OpenCV, YOLO
Automation	n8n
Background jobs	Redis + Celery
File storage	S3-compatible storage
Mobile	React Native or Flutter
Containers	Docker
Version control	Git + GitHub
Deployment	Cloud hosting such as AWS

You can use Supabase for authentication, PostgreSQL and storage when it simplifies development. Do not add every technology to every project. Select only what the architecture requires.

Day 1 — Project 1: AI-Powered WhatsApp Commerce & Retail Management SaaS

Commercial target: ₹1.5 lakh–₹3 lakh per customised implementation.

Industry: Clothing stores, jewellery shops, electronics retailers, wholesalers.

Business problem. Retail businesses receive many WhatsApp messages from customers asking about products, sizes, colours, availability, pricing and delivery. Shop owners cannot manually respond to every enquiry. Build an AI-powered commerce system that helps customers discover products and helps shop owners manage orders.

Example scenario. A clothing shop owner registers and uploads 100 products. A customer asks: "Show me red kurtis below ₹1,500." The system receives the message, understands the request, searches the actual product database, filters by colour and price, returns matching products with images and prices, lets the customer select a product, collects order information and notifies the shop owner.

Full-stack modules

- Public website: landing page, product introduction, pricing, features, registration, login.
- Admin dashboard: business overview, sales analytics, product, inventory, customer and order management, WhatsApp integration settings, AI configuration.
- Employee dashboard: customer enquiries, order processing, product availability, order status updates.
- Customer interface: WhatsApp-style conversation, product search, product details, order information, order status.

CRUD modules: businesses, users, products, categories, product variants, inventory, customers, orders, order items, conversations.

AI/ML functionality: natural-language product search, intent recognition, product recommendations, tool-calling AI agent, conversation summarisation, automated response generation. The AI must call backend tools for product prices, availability and order creation. It must never invent inventory information.

Database tables: businesses, users, roles, products, product_categories, product_variants, inventory, customers, orders, order_items, conversations, messages, integrations. Use a business_id relationship to isolate each shop's data.

API endpoints: POST /auth/register, POST /auth/login, GET /products, POST /products, PUT /products/{id}, DELETE /products/{id}, GET /customers, POST /orders, GET /orders, PUT /orders/{id}/status, POST /ai/chat, POST /webhooks/whatsapp.

Architecture: Multi-tenant SaaS + Agentic AI + WhatsApp API integration.

Data flow: Customer → WhatsApp webhook → FastAPI → AI agent → Product search tool → PostgreSQL → AI response → WhatsApp.

Three-hour MVP

Time	Task
0–20 min	Define requirements and architecture
20–45 min	Create PostgreSQL schema
45–90 min	Build backend and product APIs
90–125 min	Build admin dashboard and product CRUD
125–155 min	Integrate AI product search
155–170 min	Test order workflow
170–180 min	Deploy and record demonstration

For the MVP, use a WhatsApp-style simulator. Actual WhatsApp Cloud API onboarding and testing can be a separate production milestone.

Production additions: real WhatsApp Cloud API integration, webhook verification, multi-tenant security, payment gateway, delivery integration, inventory synchronisation, message templates and opt-in handling, audit logs, usage limits and billing.

Final deliverables: live website, admin dashboard, product CRUD, AI shopping assistant, order management, GitHub repository, architecture diagram, demo video.

Day 2 — Project 2: AI Invoice Processing & Business Document Management SaaS

Commercial target: ₹1.5 lakh–₹4 lakh.

Industry: Accounting firms, manufacturers, distributors and finance departments.

Business problem. Companies receive invoices in PDFs, scanned documents, images and emails, and employees manually enter information into accounting systems. Build a document-processing platform that extracts and validates invoice information.

Example scenario. A company uploads a supplier invoice. The application extracts supplier name, GSTIN, invoice number and date, product details, HSN codes, quantity, unit price, taxable value, GST and total amount. It then compares the invoice with the purchase order and flags any mismatch for manual review.

Full-stack modules

- Public website: landing page, product features, pricing, registration.
- Admin dashboard: company, user and supplier management, invoice analytics, processing history.
- Accountant dashboard: upload invoices, review extracted data, correct errors, export information.
- Approver dashboard: pending approvals, approved invoices, rejected invoices, mismatch reports.

CRUD modules: suppliers, invoices, purchase orders, invoice items, approval records, users, documents.

AI functionality: OCR, document classification, invoice field extraction, structured JSON output, purchase order matching, duplicate detection, mismatch explanations. Use OCR or a document vision model for extraction. Use deterministic Python logic for arithmetic and tax validation.

Database tables: companies, users, suppliers, invoices, invoice_items, purchase_orders, purchase_order_items, documents, approval_requests, audit_logs.

API endpoints: POST /documents/upload, POST /invoices/extract, GET /invoices, GET /invoices/{id}, PUT /invoices/{id}, POST /invoices/{id}/approve, POST /invoices/{id}/reject, GET /suppliers, POST /suppliers.

Architecture: Full Stack SaaS + Document AI + OCR + Human Approval Workflow.

Data flow: Upload → Document storage → OCR → Structured extraction → Validation → Database → Approval dashboard.

Three-hour MVP

Time	Task
0–20 min	Define requirements
20–45 min	Database design
45–85 min	Invoice upload API

Time	Task
85–120 min	OCR and extraction
120–145 min	Validation engine
145–165 min	Dashboard
165–180 min	Testing and deployment

Production additions: email invoice ingestion, batch document processing, purchase order reconciliation, duplicate invoice detection, multi-level approval, audit trails, accounting software integration, document retention controls.

Final deliverables: invoice upload application, OCR extraction, validation engine, approval workflow, admin dashboard, Excel export, GitHub documentation, demo video.

Day 3 — Project 3: Smart ERP, POS, Billing & Inventory Management System

Commercial target: ₹2 lakh–₹5 lakh.

Industry: Retail shops, supermarkets, distributors and wholesalers.

Business problem. Businesses often use different applications for billing, inventory, sales and suppliers. Build a unified business management platform.

Example scenario. A clothing store has three branches. The owner wants to manage products, track stock, generate invoices, manage employees, track daily sales, check profits and transfer stock between branches. The platform manages these operations centrally.

Full-stack modules

- Public website: landing page, features, pricing, registration.
- Admin dashboard: business overview, branch management, employee management, financial analytics, inventory reports.
- Cashier dashboard: POS billing, barcode scanning, customer selection, returns.
- Inventory manager dashboard: product management, purchase orders, stock transfers, supplier management.

CRUD modules: products, categories, suppliers, customers, branches, employees, purchases, sales, invoices, inventory movements.

AI functionality: sales insights, low-stock predictions, reorder suggestions, product sales trends, automated daily reports.

Database tables: businesses, users, branches, products, categories, suppliers, customers, inventory, stock_movements, sales, sale_items, purchases, purchase_items, invoices, payments. Maintain an inventory movement ledger rather than simply overwriting stock quantities.

API endpoints: GET /products, POST /products, GET /inventory, POST /sales, GET /sales, POST /purchases, GET /reports/sales, GET /reports/profit, POST /stock/transfers.

Architecture: Full Stack ERP + Transactional Database + Analytics + AI.

Data flow: POS → Backend → Transaction validation → Sales database → Inventory ledger → Analytics.

Three-hour MVP

Time	Task
0–20 min	Requirement definition
20–45 min	Database schema
45–90 min	Product and inventory APIs
90–125 min	POS billing frontend
125–150 min	Sales and stock integration
150–170 min	Reports
170–180 min	Testing and deployment

Production additions: barcode hardware integration, GST compliance validation, branch-level permissions, concurrent billing protection, offline operation strategy, returns and exchanges, accounting integrations, backup and recovery.

Final deliverables: POS system, billing application, inventory dashboard, supplier management, sales analytics, role-based access, GitHub repository, live demo.

Day 4 — Project 4: Enterprise AI Knowledge Management & RAG Platform

Commercial target: ₹1.25 lakh–₹3 lakh.

Industry: IT companies, educational institutions, manufacturers and corporate teams.

Business problem. Companies have hundreds or thousands of documents, manuals, policies and technical files, and employees waste time searching for information. Build an AI-powered company knowledge platform.

Example scenario. A manufacturing company uploads 500 technical manuals. An employee asks: "What should I do when machine number 204 displays error E17?" The application searches the relevant documents and returns an answer with citations.

Full-stack modules

- Public website: product landing page, features, pricing, signup.
- Admin dashboard: workspace management, employee management, document library, knowledge-base configuration, usage analytics.
- Employee dashboard: AI assistant, document search, chat history, saved conversations, feedback.

CRUD modules: workspaces, users, documents, document versions, knowledge bases, conversations, feedback records.

AI functionality: PDF parsing, document chunking, embedding generation, vector search, RAG, context-aware answers, source citations, semantic search, answer evaluation.

Database tables: workspaces, users, workspace_members, documents, document_chunks, embeddings, conversations, messages, feedback, document_permissions. Use PostgreSQL with pgvector.

API endpoints: POST /documents/upload, GET /documents, DELETE /documents/{id}, POST /knowledge/process, POST /chat, GET /chat/history, POST /feedback, GET /analytics/usage.

Architecture: Full Stack SaaS + RAG + Vector Search + LLM.

Data flow: Document upload → Parsing → Chunking → Embeddings → Vector database. Question → Embedding → Retrieval → Context construction → LLM → Answer with citations.

Three-hour MVP

Time	Task
0–20 min	Requirements
20–45 min	Database and vector setup
45–85 min	PDF processing
85–120 min	Embedding and retrieval
120–150 min	RAG chatbot
150–170 min	Frontend and citations
170–180 min	Testing and deployment

Production additions: permission-aware retrieval, document versioning, multi-tenant isolation, evaluation datasets, retrieval monitoring, prompt-injection protection, sensitive document controls, feedback-based improvement.

Final deliverables: document upload, RAG pipeline, AI assistant, vector search, citation-based answers, workspace dashboard, GitHub documentation, demo video.

Day 5 — Project 5: AI CRM, Lead Generation & Sales Automation Platform

Commercial target: ₹1.25 lakh–₹3 lakh.

Industry: Real estate, coaching centres, dealerships and marketing agencies.

Business problem. Companies receive leads from websites, social media and WhatsApp, and sales employees cannot always contact and follow up with every lead. Build a CRM that automates lead management.

Example scenario. A customer submits a real estate enquiry. The system creates a lead, identifies the budget and preferred location, classifies the lead, assigns it to a salesperson, creates a follow-up task, sends a WhatsApp notification and tracks the lead through the sales pipeline.

Full-stack modules

- Public website: product landing page, features, pricing, contact and registration.
- Admin dashboard: sales overview, team management, lead analytics, CRM settings.

- Sales employee dashboard: assigned leads, customer details, follow-up tasks, appointments, sales pipeline.
- Manager dashboard: team performance, lead conversion, employee activity, revenue reports.

CRUD modules: leads, customers, employees, deals, tasks, appointments, campaigns, interaction history.

AI functionality: lead qualification, lead scoring, conversation summarisation, automated follow-up suggestions, customer intent classification, appointment assistant.

Database tables: companies, users, leads, customers, deals, tasks, appointments, campaigns, interactions, messages.

API endpoints: POST /leads, GET /leads, PUT /leads/{id}, POST /leads/{id}/assign, POST /ai/qualify, POST /tasks, GET /appointments, POST /appointments, GET /reports/conversion.

Architecture: Full Stack CRM + Agentic AI + Workflow Automation.

Data flow: Lead form → CRM API → PostgreSQL → AI classification → Assignment engine → Follow-up automation.

Three-hour MVP

Time	Task
0–20 min	Requirements
20–45 min	Database
45–90 min	Lead CRUD APIs
90–120 min	CRM dashboard
120–150 min	AI lead classification
150–170 min	Follow-up workflow
170–180 min	Deployment

Production additions: WhatsApp integration, email integration, calendar integration, sales pipeline automation, duplicate lead detection, conversion analytics, team-level permissions, human-approved AI actions.

Final deliverables: CRM dashboard, lead management, AI lead qualification, sales pipeline, follow-up automation, analytics, GitHub repository, demo video.

Day 6 — Project 6: AI YouTube & Content Creation Automation Studio

Commercial target: ₹1.25 lakh–₹3 lakh.

Industry: Content creators, YouTubers, agencies and podcast studios.

Business problem. Content creators spend many hours converting long videos into short videos, preparing subtitles and creating social-media descriptions. Build an AI-powered content production application.

Example scenario. A creator uploads a 30-minute podcast. The application extracts audio, generates a transcript, identifies interesting moments, suggests short-video timestamps, generates subtitles, creates titles and descriptions, and produces a content calendar.

Full-stack modules

- Public website: product landing page, features, pricing, signup.
- Creator dashboard: video upload, project management, transcript management, clip selection, subtitle editing, video preview, export.
- Admin dashboard: users, usage statistics, processing jobs, storage management, system settings.

CRUD modules: users, projects, videos, transcripts, clips, captions, content calendars, export records.

AI functionality: speech-to-text, transcript analysis, highlight identification, clip suggestions, subtitle generation, title generation, description generation, hashtag suggestions.

Database tables: users, projects, videos, processing_jobs, transcripts, clips, captions, content_plans, exports.

API endpoints: POST /videos/upload, GET /videos, POST /videos/transcribe, POST /videos/analyse, POST /clips/generate, GET /clips, POST /captions, POST /exports.

Architecture: Full Stack SaaS + AI Pipeline + FFmpeg + Speech Recognition.

Data flow: Video upload → Object storage → Background job → FFmpeg → Speech-to-text → LLM analysis → Clip selection → Rendering → Export.

Three-hour MVP

Time	Task
0–20 min	Requirements
20–40 min	Database
40–75 min	Video upload
75–110 min	Transcription
110–140 min	AI highlight generation
140–165 min	Clip and subtitle interface
165–180 min	Testing and deployment

Use a short sample video for the MVP.

Production additions: background rendering queues, large-file uploads, video previews, subtitle editing, platform publishing integrations, user storage quotas, processing cost controls, copyright and content review workflow.

Final deliverables: video upload application, transcription pipeline, AI highlight selection, subtitle generation, content dashboard, export workflow, GitHub repository, demo video.

Day 7 — Project 7: AI Computer Vision Quality Inspection Platform

Commercial target: ₹2 lakh–₹5 lakh+.

Industry: Textile factories, automobile manufacturers, packaging and manufacturing.

Business problem. Manual quality inspection is slow and can be inconsistent. Build a computer vision platform that identifies defects in products.

Example scenario. A textile factory uploads fabric images. The model detects holes, stains, broken threads, damaged areas and colour inconsistencies, and the application creates an inspection report.

Full-stack modules

- Public website: landing page, product features, industry use cases, contact page.

- Admin dashboard: inspection analytics, employee management, model configuration, defect reports.
- Inspector dashboard: upload images, run inspection, review predictions, confirm or reject defects, generate reports.
- Manager dashboard: inspection history, defect trends, quality statistics, employee performance.

CRUD modules: products, images, inspections, defects, reports, users, inspection categories.

AI/ML functionality: image preprocessing, object detection, YOLO, OpenCV, defect classification, confidence scoring, model evaluation, human verification.

Database tables: companies, users, products, inspection_images, inspections, detected_defects, defect_categories, inspection_reports, model_versions.

API endpoints: POST /images/upload, POST /inspection/run, GET /inspection/history, GET /inspection/{id}, POST /defects/{id}/verify, GET /reports/quality, GET /models/metrics.

Architecture: Full Stack + Computer Vision + Deep Learning + ML Inference.

Data flow: Image → Backend → Preprocessing → Detection model → Prediction → Database → Inspection dashboard.

Three-hour MVP

Time	Task
0–20 min	Requirements
20–40 min	Dataset and model selection
40–80 min	Model inference
80–115 min	Image upload APIs
115–145 min	Detection dashboard
145–165 min	Inspection report
165–180 min	Testing and deployment

Use a public defect dataset and a suitable pretrained model. Clearly document the limitations of the demonstration.

Production additions: factory camera integration, custom labelled training data, model retraining, edge inference, production accuracy evaluation, real-time processing, alert management, quality assurance audit trails.

Final deliverables: image inspection application, detection model integration, defect classification, inspection dashboard, report generation, model metrics, GitHub repository, demo video.

Day 8 — Project 8: Field Service Management Mobile & Web Application

Commercial target: ₹2 lakh–₹5 lakh.

Industry: AC repair, electrical services, appliance maintenance and facility management.

Business problem. Service businesses struggle to manage customer bookings, technician schedules, job assignments and payments. Build a platform with a web admin portal and mobile technician application.

Example scenario. A customer requests AC servicing. The system creates a service ticket, stores customer details, finds an available technician, assigns the job, sends an appointment confirmation, lets the technician update job status, captures repair photographs, generates an invoice and records payment.

Full-stack modules

- Public website: service information, booking form, pricing, contact.
- Customer portal: service booking, booking history, technician status, invoices, feedback.
- Admin dashboard: customers, technicians, job assignments, scheduling, payments, reports.
- Technician mobile app: login, assigned jobs, customer information, job status, photos, parts used, job completion.

CRUD modules: customers, technicians, service categories, service tickets, appointments, parts, invoices, payments, feedback.

AI functionality: service request classification, technician assignment recommendations, customer chatbot, appointment reminders, route suggestions, automated job summaries.

Database tables: users, customers, technicians, services, service_tickets, appointments, job_assignments, service_parts, invoices, payments, feedback.

API endpoints: POST /bookings, GET /bookings, POST /jobs/assign, PUT /jobs/{id}/status, GET /technicians, POST /jobs/{id}/photos, POST /invoices, POST /payments.

Architecture: Full Stack Web + Mobile + FastAPI + PostgreSQL + AI Automation.

Data flow: Customer app → Booking API → Assignment engine → Technician app → Job update → Invoice → Customer notification.

Three-hour MVP

Time	Task
0–20 min	Requirements
20–45 min	Database
45–80 min	Booking and job APIs
80–120 min	Admin dashboard
120–145 min	Technician interface
145–165 min	Assignment workflow
165–180 min	Testing and deployment

For the three-hour version, a responsive mobile web interface can serve as the technician app. Build a native app as a subsequent milestone.

Production additions: native mobile applications, offline job updates, GPS permissions, route optimisation, push notifications, payment integration, technician attendance, service history.

Final deliverables: customer booking portal, admin dashboard, technician interface, job management, invoice workflow, AI classification, GitHub repository, demo video.

Day 9 — Project 9: AI Finance, Payment Reconciliation & Expense Management SaaS

Commercial target: ₹1.5 lakh–₹4 lakh.

Industry: SMEs, distributors, accounting firms and agencies.

Business problem. Companies struggle to match customer payments against invoices and identify overdue or unmatched transactions. Build a financial operations platform.

Example scenario. A company uploads invoice records, customer payment records and a bank statement CSV. The system matches payments against invoices and identifies fully paid invoices, partially paid invoices, unmatched payments, duplicate transactions and overdue invoices.

Full-stack modules

- Public website: product landing page, features, pricing, contact.
- Admin dashboard: financial overview, user management, customer accounts, supplier accounts, financial reports.
- Accountant dashboard: invoice management, bank statement upload, transaction reconciliation, expense management, export.
- Approver dashboard: pending reconciliations, manual matching approval, exception reports.

CRUD modules: customers, suppliers, invoices, transactions, expenses, accounts, reconciliation records, approval records.

AI functionality: expense categorisation, transaction matching assistance, duplicate detection, unmatched transaction explanations, cash-flow summaries, payment reminders. Use deterministic matching for confirmed financial reconciliation. AI should assist with ambiguous cases, not silently approve them.

Database tables: companies, users, customers, suppliers, accounts, invoices, invoice_items, transactions, expenses, reconciliation_matches, approvals.

API endpoints: POST /invoices, POST /transactions/import, POST /reconciliation/run, GET /reconciliation/results, POST /reconciliation/{id}/approve, GET /expenses, POST /expenses, GET /reports/cashflow.

Architecture: Full Stack Financial SaaS + Data Processing + Matching Engine + AI.

Data flow: CSV import → Data normalisation → Matching engine → Exception identification → Human approval → Financial dashboard.

Three-hour MVP

Time	Task
0–20 min	Requirements
20–45 min	Database
45–80 min	CSV upload and parsing

Time	Task
80–115 min	Matching engine
115–145 min	Reconciliation dashboard
145–165 min	AI-assisted explanations
165–180 min	Testing and deployment

Production additions: accounting integrations, secure data import, audit trails, approval workflows, bank integrations where supported, reconciliation rules, financial reporting, data retention and backup.

Final deliverables: finance dashboard, invoice management, transaction importer, reconciliation engine, exception reporting, expense management, GitHub repository, demo video.

Day 10 — Project 10: AI Demand Forecasting & Smart Inventory Prediction SaaS

Commercial target: ₹2 lakh–₹5 lakh+.

Industry: Retail chains, supermarkets, distributors and e-commerce.

Business problem. Businesses often purchase too much stock or run out of popular products. Build an ML-powered demand forecasting platform.

Example scenario. A retailer uploads six months of historical sales, and the system predicts expected demand for the coming weeks.

Product	Current stock	Predicted demand	Recommendation
Red Kurti	20	45	Purchase 25
Blue Saree	50	30	No immediate purchase
Black Shirt	10	40	Purchase 30

These are illustrative figures. Actual recommendations must account for lead time, safety stock and existing purchase orders.

Full-stack modules

- Public website: product landing page, features, pricing, contact.
- Admin dashboard: business overview, branch management, user management, inventory analytics.
- Inventory manager dashboard: product management, stock records, reorder recommendations, supplier information.
- Analyst dashboard: historical sales, forecast visualisations, model performance, forecast comparisons.

CRUD modules: products, categories, branches, sales records, inventory, suppliers, purchase orders, forecasts, reorder recommendations.

AI/ML functionality: data cleaning, feature engineering, time-series forecasting, Scikit-learn, XGBoost, seasonal analysis, stock-out risk, reorder recommendations, forecast accuracy evaluation. Compare a simple baseline model against more advanced models. Do not claim forecasting accuracy without testing.

Database tables: businesses, users, products, branches, sales, sale_items, inventory, suppliers, purchase_orders, forecasts, forecast_metrics, reorder_recommendations.

API endpoints: POST /sales/import, GET /sales/history, POST /forecast/train, POST /forecast/predict, GET /forecast/results, GET /inventory, GET /inventory/recommendations, POST /purchase-orders, GET /reports/forecast.

Architecture: Full Stack SaaS + Data Engineering + Machine Learning + Predictive Analytics.

Data flow: Sales CSV → Data validation → Feature engineering → ML model → Forecast results → Reorder engine → Dashboard.

Three-hour MVP

Time	Task
0–20 min	Requirements
20–45 min	Database
45–80 min	CSV upload and preprocessing
80–115 min	ML model
115–145 min	Forecast API and dashboard
145–165 min	Reorder recommendation

Time	Task
165–180 min	Testing and deployment

Production additions: automatic data ingestion, model retraining, forecast monitoring, multi-branch support, seasonal adjustment, stock lead-time calculations, forecast confidence intervals, inventory system integration.

Final deliverables: inventory dashboard, sales data importer, forecasting model, forecast API, reorder recommendation engine, analytics, GitHub repository, demo video.

Part 3 — Daily Project Execution Checklist

Use this checklist for every project.

Before development

- [ ] Understand the business problem.
- [ ] Identify the target customer.
- [ ] Identify user roles.
- [ ] Define the MVP scope.
- [ ] Select the architecture.
- [ ] Design database tables.
- [ ] Define API endpoints.
- [ ] Prepare the GitHub repository.

During development

- [ ] Build the frontend.
- [ ] Build backend APIs.
- [ ] Connect the database.
- [ ] Implement CRUD.
- [ ] Implement authentication.
- [ ] Implement role permissions.
- [ ] Integrate AI/ML.
- [ ] Connect automation.
- [ ] Validate inputs.

- [ ] Handle errors.

Before completion

- [ ] Test all important workflows.
- [ ] Test role-based access.
- [ ] Test API responses.
- [ ] Check database persistence.
- [ ] Deploy frontend.
- [ ] Deploy backend.
- [ ] Configure environment variables.
- [ ] Write README.
- [ ] Create architecture diagram.
- [ ] Create database ER diagram.
- [ ] Document APIs.
- [ ] Record a demonstration.
- [ ] Save GitHub and live application links.

Part 4 — GitHub Repository Template

Use this folder structure as a starting point for projects with a separate frontend and backend.

```
project-name/
│
├── frontend/
│   ├── app/
│   ├── components/
│   ├── hooks/
│   ├── services/
│   ├── types/
│   └── public/
│
├── backend/
│   ├── app/
│   │   ├── api/
│   │   ├── models/
│   │   ├── schemas/
```

```
|   |   ├── services/
|   |   ├── agents/
|   |   ├── ml/
|   |   ├── core/
|   |   └── main.py
|   ├── tests/
|   └── requirements.txt
|
├── database/
|   ├── migrations/
|   └── schema.sql
|
├── docs/
|   ├── architecture.md
|   ├── data-flow.md
|   ├── database-erd.md
|   └── api-documentation.md
|
├── deployment/
|   ├── Dockerfile
|   └── docker-compose.yml
|
├── .env.example
├── .gitignore
└── README.md
```

Adjust this structure for projects that need separate ML services, mobile apps or background workers. Never commit real API keys, passwords or production credentials.

Part 5 — Project Review and Commercial Evaluation

When you finish each project, provide:

1. GitHub repository URL.
2. Live application URL.
3. Demo video.
4. Test account credentials using a dedicated demo account.
5. Architecture diagram.
6. README.
7. Screenshots.

8. Explanation of the business problem and solution.

I will review your actual implementation.

Evaluation scoring

Evaluation area	Weight
Business value and ROI	20
Architecture and scalability	15
AI/ML implementation	15
Backend and database	15
Frontend and user experience	10
Security and reliability	10
Testing and documentation	10
Deployment and demonstration	5
Total	**100**

Commercial assessment

For every completed project, I will provide: technical quality score, architecture review, full-stack code review, AI/ML review, security gaps, missing features, production readiness, potential implementation pricing, client types that could buy it, and improvements needed to justify ₹1 lakh or more.

The commercial assessment will consider implementation scope, integration complexity, business impact, deployment, support and customer requirements.

Part 6 — Your 10-Day Completion Tracker

Day	Project	Planned completion
Day 1	WhatsApp Commerce SaaS	Today
Day 2	Invoice Processing SaaS	Tomorrow

Day	Project	Planned completion
Day 3	ERP, POS & Inventory	Day 3
Day 4	Enterprise RAG Platform	Day 4
Day 5	AI CRM	Day 5
Day 6	YouTube Automation Studio	Day 6
Day 7	Computer Vision Inspection	Day 7
Day 8	Field Service Application	Day 8
Day 9	Finance Reconciliation SaaS	Day 9
Day 10	Demand Forecasting SaaS	Day 10

Important: The three-hour daily target is for a working MVP. Additional production milestones may take longer. A completed MVP is not automatically a production-ready enterprise product.

Final Goal

At the end of these ten days, your portfolio should demonstrate: 10 business-oriented software products; full stack development; Python backend development; React and Next.js; database design; REST API development; authentication and authorization; AI and machine learning; generative AI; agentic AI; RAG; computer vision; workflow automation; WhatsApp API integration; mobile application development; cloud deployment; Docker and CI/CD; GitHub and technical documentation.

Your development principle: Do not build a project just to say you have built an AI application. Build a product that solves a real business problem, demonstrates sound software architecture, and can be evaluated by a potential employer or customer.

Start with Day 1: AI-Powered WhatsApp Commerce & Retail Management SaaS. Complete its MVP, test it, deploy it, and bring the repository and live link for review before moving to Day 2.